

EPISÓDIO 6

A pirâmide moderna de qualidade de dados

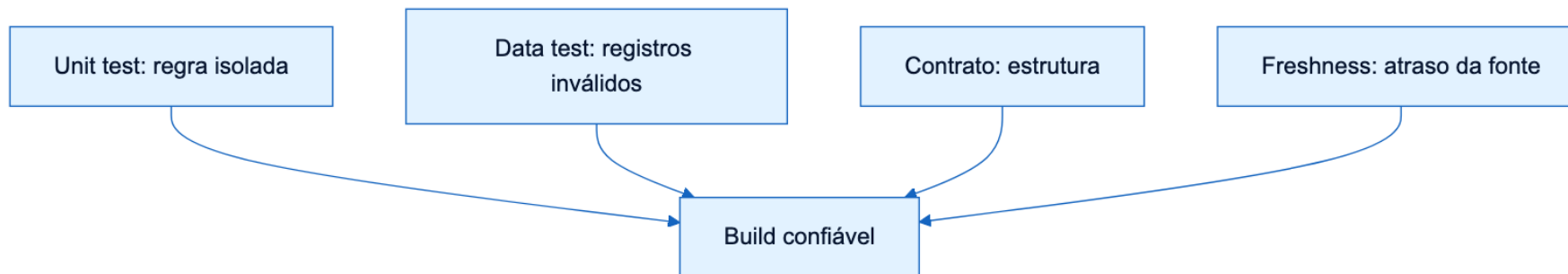
Cada risco pede um mecanismo de detecção diferente

Checkpoint 06 • DuckDB local • sem cloud



not_null pode passar enquanto um prazo devolve menos cinco dias.

A pergunta que guia a aula



Ao final, você consegue...

01

Separar cinco
mecanismos



02

Isolar uma regra
funcional



03

Testar freshness
sem depender da
data

Cinco perguntas, cinco mecanismos

Unit, contrato, data test, freshness e observabilidade cobrem riscos distintos.

- Lógica
- Estrutura
- População
- Atraso
- Comportamento



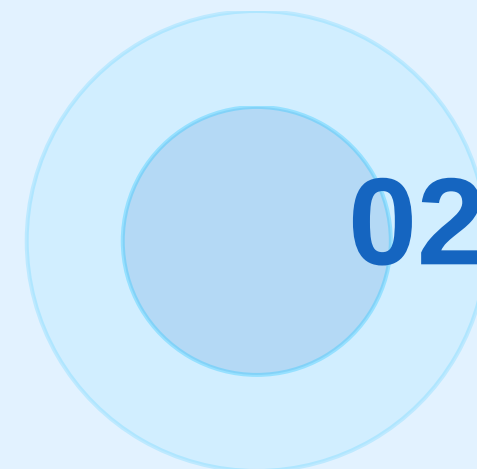
Cinco perguntas, cinco mecanismos



Unit test captura o bug conhecido

Compra em 1º e entrega em 5 de janeiro devem produzir quatro dias positivos.

- Entrada mínima
- Saída esperada
- Regressão determinística



Unit test captura o bug conhecido

Fixture

- Entrada mínima
- Saída esperada

4 dias

- Regressão determinística

O exemplo oficial mostra execução isolada

Unit tests validam lógica SQL antes de materializar toda a população.

- Inputs estáticos
- Modelo selecionado
- Falha localizada



O exemplo oficial mostra execução isolada

Unit tests validam lógica SQL antes de materializar toda a população.

```
(dbt_1.8) $ dbt build --select hello_world

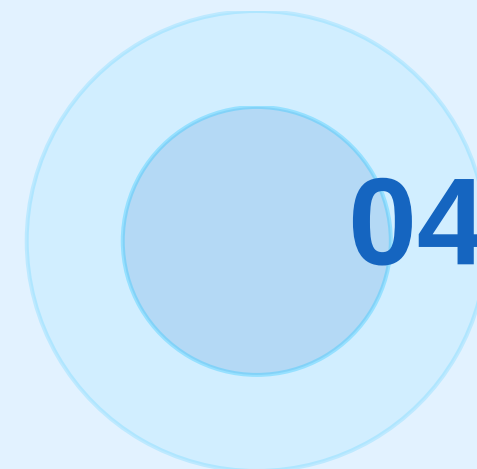
21:43:54 Running with dbt=1.8.0-rc1
21:43:54 Registered adapter: postgres=1.8.0-rc1
21:43:55 Found 6 models, 3 analyses, 411 macros, 3 unit tests
21:43:55
21:43:55 Concurrency: 5 threads (target='postgres')
21:43:55
21:43:55 1 of 2 START unit_test hello_world::test_hello_world ..... [RUN]
21:43:55 1 of 2 PASS hello_world::test_hello_world ..... [PASS in 0.17s]
21:43:55 2 of 2 START sql view model dbt_dbeatty.hello_world ..... [RUN]
21:43:55 2 of 2 OK created sql view model dbt_dbeatty.hello_world ..... [CREATE VIEW in 0.09s]
21:43:55
21:43:55 Finished running 1 unit test, 1 view model in 0 hours 0 minutes and 0.55 seconds (0.55s).
21:43:55
21:43:55 Completed successfully
21:43:55
21:43:55 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2
```

Exemplo oficial dbt Labs — Apache 2.0

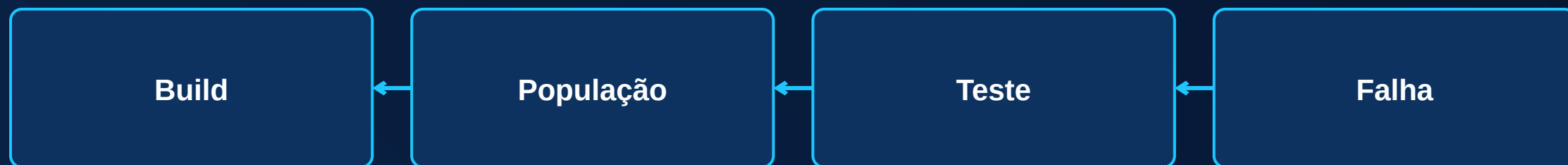
Data tests procuram violações reais

A população materializada pode falhar mesmo quando o exemplo unitário passa.

- unique e not_null
- accepted_values
- Regra delivered_revenue



Data tests procuram violações reais



Freshness responde se a fonte chegou

Uma fixture recente passa; a mesma carga envelhecida deve falhar.

- loaded_at_field
- SLA explícito
- Teste positivo e negativo



Freshness responde se a fonte chegou



Recente



Warning



Error

Demonstração — execute

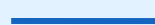
```
python course.py checkpoint 06
```

Copie exatamente como aparece. O launcher escolhe o ambiente correto.

Demonstração — observe

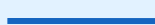
01

Unit test do intervalo



02

25 data tests



03

**Freshness recente e
envelhecida**

RESULTADO ESPERADO

45 PASS • 1 NO-OP • 0 erros

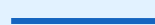
Checkpoint 06 • evidência reproduzível



O mesmo recurso afeta três decisões

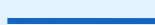
01

Governança: risco e owner



02

FinOps: controle na etapa barata



03

IA: interfaces testadas

+4 dias

resultado esperado do bug de data

Mini-case reproduzido no
laboratório.

Limitações que precisam permanecer visíveis

- Freshness não mede volume
- Teste pesado pode custar
- Alerta sem runbook vira ruído

RADAR

Engines novas antecipam parse; não substituem testes, freshness ou SLO.



Novidade observada $\neq$ recomendação de produção

PRÓXIMO PASSO

Escreva o menor unit test que reproduz um bug possível.

Checkpoint 06 • roteiro completo no repositório